LINUX / SYSTEMS

Containers vs VMs Visual Guide

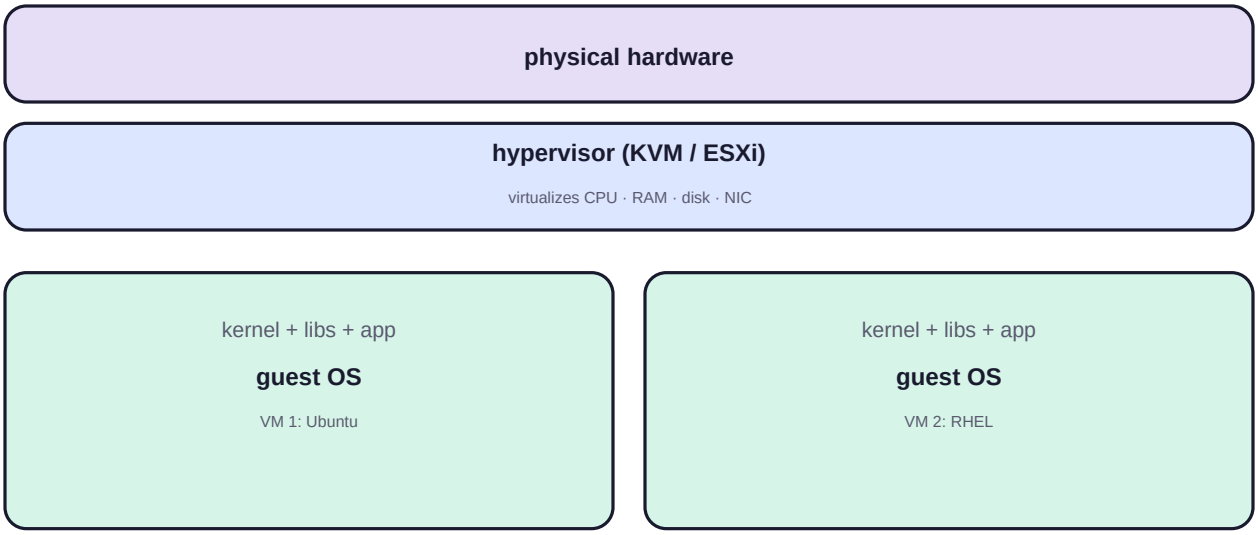
Isolation levels, overhead,
when each wins — and the middle ground.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

VM Architecture: Hardware in Software

A hypervisor virtualizes hardware; each VM runs a full OS.



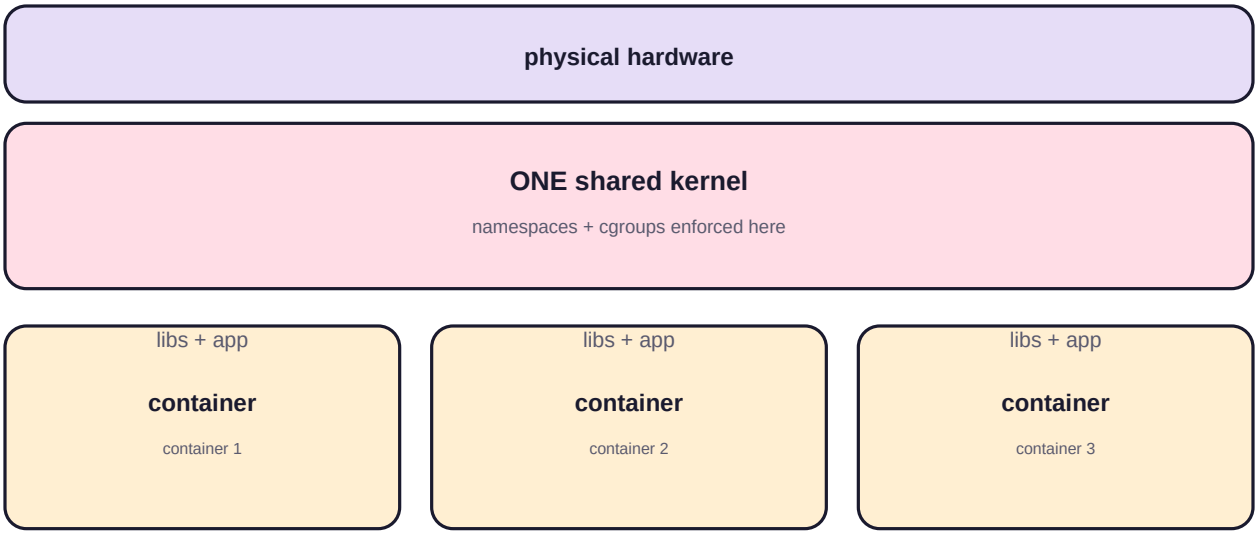
Every VM carries its own kernel — strong isolation, real overhead per guest.

Type 1 hypervisors (KVM, ESXi) run on bare metal; Type 2 (VirtualBox) run inside a host OS. Production servers use Type 1.

STEP 2/6

Container Architecture: One Kernel, Many Views

Namespaces + cgroups partition a single OS.

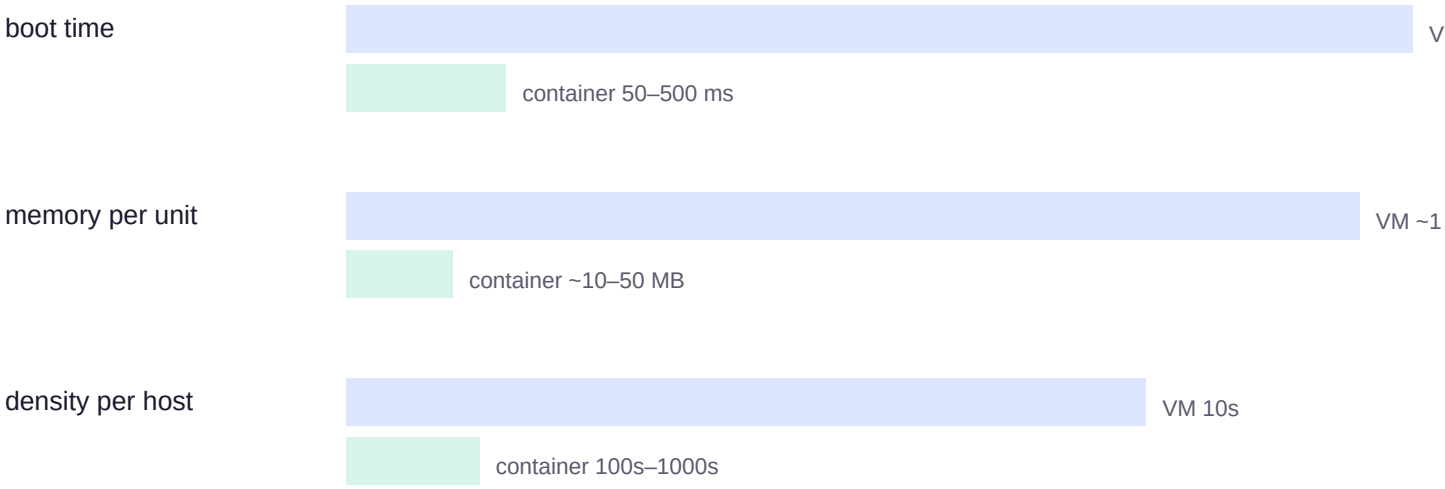


No guest kernels — containers are processes with isolation flags. Milliseconds to start.

STEP 3/6

Overhead Comparison

What the extra kernel(s) cost you.



Bars are illustrative, not benchmarks — measure your own workload. The DIRECTION is always the same: containers are 10–100x lighter per unit.

STEP 4/6

Isolation & Security Compared

Stronger walls cost more concrete.

	VM	Container
isolation boundary	hypervisor + own kernel	kernel namespaces
kernel exploit escapes	to hypervisor only	to HOST kernel
tenants per host	untrusted OK	trust the kernel together
live migration	mature	hard (CRIU)
custom kernel/modules	yes	no — host kernel
startup	slow	instant

- Rule of thumb: hostile or mutually-untrusted tenants → VMs. One team, one org → containers.
- Containers share the kernel — a kernel CVE is a container-escape CVE. Patch hosts fast.
- Defense in depth for containers: read-only rootfs, drop capabilities, seccomp, user namespaces.

STEP 5/6

The Middle Ground: Kata, gVisor, Firecracker

VM-grade isolation at container speed.

Kata Containers

each pod in a■light VM

gVisor

user-space kernel■(Sentry) shim

Firecracker

microVMs:■125ms boot

Tech	Isolation trick	Used by
Kata	lightweight QEMU VM per pod	regulated multi-tenant K8s
gVisor	intercepts syscalls in userspace	GCP serverless
Firecracker	KVM microVM, minimal device model	AWS Lambda/Fargate

Pick the middle when containers are right operationally but the threat model needs VM walls — e.g. running customer code in your SaaS.

STEP 6/6

Decision Guide

Match the tool to the job.

- ✓ Need different OS kernels (Windows + Linux)? → VMs, no contest
- ✓ Running untrusted third-party code? → VMs (or Kata/Firecracker)
- ✓ Microservices, CI jobs, dev envs, dense packing? → containers
- ✓ Need bare-metal speed with isolation? → containers + strict seccomp/caps
- ✓ Compliance demands kernel separation? → VMs, document it
- ✓ When unsure: containers by default, VMs where the threat model says so

STEP ✓/✓

Key Takeaways

Containers vs VMs

- VMs virtualize hardware — each guest carries a full kernel: strong isolation, heavy overhead.
- Containers partition one kernel with namespaces + cgroups: millisecond starts, shared-kernel risk.
- Overhead gap is 10–100x per unit — density is containers' superpower.
- Untrusted tenants → VMs; one org's services → containers; harden both.
- Middle ground (Kata, gVisor, Firecracker) buys VM-grade walls at container speed.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev